

NEPVERSA: A Version-Aware Nepali Statutory Retrieval Benchmark Derived from Amendment Footnotes

NIKHIL BHAGAT, Independent Researcher, Nepal

Retrieval-augmented generation systems assume that newer documents supersede older ones. Statutory law violates this assumption: when an Act is amended, the earlier text remains authoritative for events preceding the amendment. Recent benchmarks show that retrieval over a static consolidated corpus returns real but inapplicable provisions, but they cover high-resource languages with strong retrievers. Nepali has neither: it has no standard information retrieval benchmark, and published results show lexical retrieval outperforming multilingual dense retrieval on Nepali legal text. Building a version-aware Nepali corpus appears blocked: Gazette notices are scanned images and official Act PDFs extract to corrupted text. We show that the amendment record is nonetheless recoverable, from numbered footnotes carried inline in consolidated Acts, which name the amendment and its operation (insertion, substitution, or repeal) at clause granularity. We present **NEPVERSA**, the first version-aware Nepali statutory benchmark, whose candidate gold is derived mechanically from these published statements and scored by deterministic nuggets rather than by a language-model judge. Footnote markers are read from the published HTML, where each of the 119 markers is either attached to a unit or reported; a Markdown-only reading of the same pages misses 26 amended units and invents 6. Our central finding is that an approximately correct parser yields a corpus that appears entirely correct, because a missed amendment is indistinguishable from a provision never amended. The release covers 5 Acts, 455 provisions and 181 items, each checked against its source footnote by the author; a legal-expert audit remains outstanding.

Additional Key Words and Phrases: legal information retrieval, low-resource languages, Nepali, temporal reasoning, benchmark construction, retrieval-augmented generation

1 Introduction

Retrieval-augmented generation (RAG) systems answer questions by retrieving documents and conditioning a language model on them [Lewis et al. 2020]. Recent work has made retrieval adaptive: FLARE triggers retrieval when token probabilities fall [Jiang et al. 2023], Adaptive-RAG routes questions by predicted complexity [Jeong et al. 2024], IRCot interleaves retrieval with chain-of-thought reasoning [Trivedi et al. 2023], and reinforcement-learned agents issue their own queries during reasoning [Jin et al. 2025; Yao et al. 2023].

These methods share an assumption that is invisible until it breaks: that a newer document supersedes an older one. In an encyclopedia this holds. In law it does not. When a legislature amends an Act, the earlier text remains the correct law for events that occurred before the amendment took effect. A question about a 2078 dispute must be answered from the 2078 text, and a system that retrieves today’s consolidated version returns a real provision that does not govern.

Recent work has begun to measure this failure. Cymbler et al. [2026] show that on French tax law, retrieval over a single current snapshot never once returned the article text that applied at the date asked about (0%), and instead cited real provisions that did not apply. Sobhani et al. [2026] find that models resolve a version from an implicit date in 59.7% of cases but detect that a supplied version does *not* govern in only 26.7%. Prior et al. [2026] report the same pattern in German statutory question answering and argue that whether a provision was in force must filter the candidates, not merely reorder them.

Every one of these results comes from a high-resource language with a strong retriever. Nepali is neither.

1.1 Why Nepali is the harder case

Nepali is spoken by over 26 million people and has no standard information retrieval benchmark at all [Acharya and Bal 2026]. What evidence exists suggests that the dense retrievers underpinning modern RAG perform poorly on it. Wagle et al. [2026], introducing the first RAG system for Nepali legal question answering, report **BM25 at P@1 0.91 against multilingual-E5-large at 0.75**, a lexical baseline beating a neural retriever by 16 points. Our own independent measurements on a different Nepali legal corpus reproduce the direction of that result (the Results section).

This matters for the temporal problem because every published mitigation assumes a retriever strong enough to surface candidate versions before a filter or an agent can choose between them. **If first-stage recall is weak, version selection never gets the chance to fail properly. It fails earlier, and for a different reason.** Whether the findings of Cymbler et al. [2026] and Sobhani et al. [2026] survive that regime is an open empirical question, and it cannot be asked without a version-aware corpus in such a language.

No such corpus exists. Existing Nepali legal NLP works over case law with no statutory versioning [Wagle et al. 2026] or provides an assistant evaluated on answer similarity with no temporal dimension at all [Thapa et al. 2026].

1.2 The obstacle, and the way around it

Building one is harder than it sounds, for a reason specific to how Nepal publishes law. A validity window is normally derived from gazette commencement notices. In Nepal those notices are overwhelmingly published as scanned images: of 53 gazette documents we examined, **50 contained no machine-readable text**. The consolidated Act PDFs on the Nepal Law Commission website are worse than unhelpful. All 61 extract to orthographically impossible text (मिति → मममि) because the typesetting pipeline writes incorrect Unicode codepoints into real Unicode fonts. The documents look correct on screen and cannot be quoted.

Our contribution begins with the observation that the amendment record is already present in a different place: **inside the consolidated statutory text itself**, as numbered footnotes attached to individual clauses.

- 1(d1) Avail credit or advance facilities by the Chief Executive Officer ...
 2(e) Re-avail or re-provide loans from or by another Bank ...

1 Added by the First Amendment.

2 Amended by the First Amendment.

Each marker names the amendment and, through its legend, the operation it performed (insertion, substitution, or repeal) **at clause granularity**. This is amendment provenance published as part of the law, and it can be extracted deterministically. Ground truth for a version-aware benchmark therefore does not require expert annotation of what changed: the statute states it.

Figure 1 shows what that yields for a single real provision.

1.3 Contributions

1. **A method for deriving provision versions from amendment footnotes** in consolidated statutes, with an explicit account of what the source supports and what it does not. Insertions and repeals yield fully determined validity windows; substitutions yield a determined *post*-state and an undetermined prior text, and we mark this rather than imputing it.
2. **NEPVERSA**, the first version-aware Nepali statutory benchmark, with *candidate* gold derived mechanically from published amendment footnotes and scored by deterministic

Section 2 Definitions

बैंडकड्डिग कसूर तथा सजाय ऐन (Banking Offence and Punishment Act)



Point-in-time question (T2)

"What did section 2(f) require as of a date before the First Amendment?"

Correct: Version 1. Wrong-version answer: Version 2, quoted accurately and dated wrongly.

Fig. 1. A Substituted Clause, Its Two Versions, and the Question They License. Section 2 of the Banking Offence and Punishment Act, 2064, whose clause (f) was substituted by the First Amendment. The substitution divides the provision’s life into two versions. A question anchored before the amendment is answerable only from Version 1, and a system that retrieves Version 2 can quote it accurately, cite it correctly, and still be wrong, because it is wrong about the date rather than about the text. The provision, its operation and its amendment ordinal are read from the harvested corpus; only the layout is drawn.

nuggets rather than by an LLM judge, following [Cymbler et al. \[2026\]](#), who argue that a model used as a judge would carry the very temporal error it is asked to detect.

3. **An extraction-accuracy evaluation** against the published HTML markup, together with a cross-signal validator. We report a finding we consider central for anyone building such a corpus: **a footnote parser that is approximately correct produces a corpus that appears entirely correct**, because a missed amendment is indistinguishable in the output from a provision that was never amended.
4. **A characterisation of the weak-retriever regime** in Nepali legal retrieval, with re-derived measurements, and an account of what it implies for the transferability of published version-aware RAG results.

All code, extraction rules, benchmark items and validation findings are released. Because the licence status of the underlying statutory text is unresolved (the Ethical Considerations section), we release identifiers, offsets, annotations and a rebuild script rather than the source text.

2 Related Work

2.1 Adaptive and agentic retrieval

A substantial line of work makes retrieval conditional. FLARE triggers retrieval when generated-token probabilities fall below a threshold [[Jiang et al. 2023](#)]. Adaptive-RAG trains a classifier that routes each question to no retrieval, single-step retrieval, or multi-step retrieval according to predicted complexity [[Jeong et al. 2024](#)]. IRCOT interleaves chain-of-thought reasoning with retrieval so that each reasoning step can issue a new query [[Trivedi et al. 2023](#)], and reinforcement-learned agents such as Search-R1 learn query policies directly [[Jin et al. 2025](#)]. ReAct established the general pattern of interleaving reasoning with tool calls [[Yao et al. 2023](#)].

These methods were developed and evaluated on English open-domain benchmarks with strong dense retrievers. The present work does not propose a new retrieval policy. It supplies the resource needed to ask whether the assumptions behind such policies hold in a low-resource, version-sensitive setting.

2.2 Legal retrieval benchmarks

Statutory article retrieval was given its first large expert-labelled resource by BSARD: more than 1,100 questions in French, each linked by jurists to the relevant provisions among more than 22,600 articles of Belgian legislation [Louis and Spanakis 2022]. Case-law retrieval has since received larger resources: CLERC for U.S. citation retrieval and retrieval-augmented analysis [Hou et al. 2025], and LePaRD for predicting the precedential passage a judge cites [Mahari et al. 2024]. LegalBench measures legal reasoning across 162 expert-designed tasks [Guha et al. 2023], and LegalBench-RAG traces those tasks back to the smallest supporting passages so that retrieval can be scored on its own, over 6,858 expert-annotated query–answer pairs [Pipitone and Hour Alami 2024].

As described by their authors, each of these resources evaluates retrieval against one snapshot of the law. The question a practitioner asks about a past transaction, namely which text governed *then*, is outside what a snapshot can represent, since the snapshot holds a single version of each provision. This is the gap the version-aware resources below, and NEPVERSA, address.

2.3 Time-sensitive question answering

Outside law, answers that change with time have been studied as a distinct QA problem. SituatedQA shows that roughly 16.5% of NQ-Open questions have context-dependent answers and that models trained on past data lose about 15 points when asked in the present, even with an updated evidence corpus [Zhang and Choi 2021]. TimeQA builds time-sensitive questions from time-evolving Wikidata facts [Chen et al. 2021], TempLAMA probes language models for facts with expiry dates [Dhingra et al. 2022], and StreamingQA evaluates adaptation over 14 years of time-stamped news [Liška et al. 2022]. These datasets treat the newest evidence as authoritative for the present. Statutory questions invert that: the newer text is authoritative only from its commencement, and a question about an earlier period must be answered from the older text even though the newer one exists.

2.4 Temporal and version-aware legal retrieval

Three 2026 results establish that temporal validity is a distinct failure mode rather than a ranking nuisance.

Cymbler et al. [2026] assemble 32,436 dated versions of French tax-code articles covering 1938–2031. Retrieval against only the current text never returned the version that applied at the date asked about (0%), yet answered with confident citations to provisions that were real and inapplicable. When every version was indexed, a retriever trained end to end reached 98.3% strict accuracy, which places the failure in how the corpus was built rather than in what the model can do. They grade answers against atomic nuggets and refuse a model-based judge, reasoning that such a judge shares the temporal blind spot under test; we follow the same choice.

Sobhani et al. [2026] introduce TIDE, 3,050 expert-verified question–answer pairs over 644 Bangladeshi customs instruments issued between 1969 and 2025, and name **version resolution** as the central challenge. Their sharpest result is asymmetric: models resolve a version from an implicit date in 59.7% of cases but recognise that a supplied version does *not* govern in only 26.7%. Detection of non-applicability is much harder than selection of the applicable version, which motivates our treatment of abstention as a first-class measured outcome.

Prior et al. [2026] study German statutory question answering and separate two failure modes (post-cutoff staleness and recency bias), and conclude that a provision’s period of validity has to be enforced as a condition on the answer rather than weighed as one signal among others. Fan et al.

[2026] approach the same problem from the training side, learning temporal consistency through reinforcement over temporally-indexed data spanning multiple amendment periods.

Our work differs in jurisdiction, language, and in what it supplies. TIDE and FiscalQA Pro are benchmarks *plus* model evaluations in languages with mature retrieval infrastructure. We supply the missing resource for a language that has none, together with the extraction method that makes such a resource obtainable where the conventional source (gazette commencement notices) is unusable.

2.5 Nepali legal and low-resource NLP

Wagle et al. [2026] present the first RAG system for Nepali legal question answering, using case law from the Nepal Kanun Patrika archive. Two of their findings bear directly on this work. First, **BM25 reaches P@1 0.91 while multilingual-E5-large reaches 0.75**, a lexical method outperforming a neural one by a wide margin, which is the inverse of the English pattern the adaptive retrieval literature assumes. Second, their corpus is case law only: no statutes, no amendments, no versioning.

Thapa et al. [2026] introduce NepKANUN, a fine-tuned RAG assistant for Nepali legal queries evaluated by BERTScore (F1 0.82 / 0.77 / 0.71 across question difficulty). It is a system rather than a resource, and carries no temporal dimension.

Acharya and Bal [2026] argue that Nepali lacks any standardised information retrieval benchmark and propose building one covering monolingual, cross-lingual, and code-mixed retrieval, noting that Nepali is a compelling testbed for morphology-aware retrieval. Their paper establishes the gap; it does not close it.

On the modelling side, NepBERTa pre-trains a Nepali encoder on a large general corpus [Timilsina et al. 2022], and NepLEGiT pre-trains a small generative model of roughly 30 million parameters on a legal corpus of about four million tokens drawn from Nepal’s constitution, its civil and criminal codes and its regulations [Raut et al. 2026]. NepLEGiT is the closest prior use of Nepali statutory text; it treats the statutes as a pre-training corpus, with no representation of amendments or of the period in which a provision applied.

Comparable efforts in other low-resource legal settings, such as Urdu [Faisal and Yousaf 2024], likewise build question-answering resources without a versioning dimension.

2.6 Nugget evaluation and abstention

Scoring an answer by the atomic facts it must contain originates in the TREC Question Answering track; the TREC 2024 RAG track revived it, and its AutoNuggetizer framework uses a language model both to create nuggets and to assign them to answers, reporting strong correlation with a mostly manual assessment [Pradeep et al. 2024]. NEPVERSA keeps the nugget unit but removes the model from both steps: nuggets are generated from amendment footnotes and matched by string rules, following the argument of Cymbler et al. [2026] that a model judge inherits the temporal bias under measurement.

Abstention as a scored outcome follows SQuAD 2.0, which added over 50,000 adversarially written unanswerable questions so that a system must decide when no answer is supported [Rajpurkar et al. 2018]. In a version-aware benchmark unanswerability is not written in; it follows from the law. A clause does not exist before the amendment that inserted it, so the correct answer to a question about that period is a refusal.

To our knowledge no prior work constructs a version-aware Nepali statutory corpus, and none uses Bikram Sambat validity windows. Systematic searches of arXiv for Nepali combined with amendment or statutory corpus terms, and for Bikram Sambat, returned no relevant prior work (the Related Work section).

3 Method

3.1 Problem formulation

We first state the retrieval task precisely, because version-aware retrieval differs from standard retrieval in what counts as a correct answer rather than in how candidates are scored.

Let $\mathcal{P}$ be the set of statutory units (sections, sub-sections and clauses) in the corpus, and let each unit $u \in \mathcal{P}$ carry an ordered sequence of amendment events. Writing a_0 for enactment and $a_1, \dots, a_k$ for the successive amendments that touch u , the unit’s history partitions into versions

$$\Pi(u) = \{(\nu_i, [a_i, a_{i+1})) : 0 \leq i \leq k\}, \quad a_{k+1} = \infty,$$

where ν_i is the text in force over the half-open window $[a_i, a_{i+1})$. The windows are half-open by construction, so for any reference point τ exactly one version governs:

$$|\{i : \tau \in [a_i, a_{i+1})\}| = 1.$$

A closed upper bound would satisfy two versions at $\tau = a_{i+1}$, which is precisely the boundary a point-in-time question is most likely to probe.

A query is a pair $q = (\text{text}, \tau)$, and the governing version is

$$g(q) = \nu_i \text{ such that } \tau \in [a_i, a_{i+1}).$$

A system returns a response r . Following [Sobhani et al. \[2026\]](#) we score two properties that are **never averaged into one number**:

$$\text{MC}(r, q) = \mathbb{1}[r \text{ satisfies the content nuggets of } g(q)],$$

$$\text{TC}(r, q) = \begin{cases} 1, & g(q) \neq \emptyset, \\ \mathbb{1}[r \text{ abstains}], & g(q) = \emptyset. \end{cases}$$

Whether r cites $g(q)$ is checked inside MC, by a citation nugget; TC isolates the one temporal judgement the benchmark adds, namely whether the system refuses when nothing governed at τ . The failure this benchmark exists to expose is the answer that fails TC: a fluent response that quotes a real provision, accurately and with a correct citation, for a point in time at which that provision did not govern. A single averaged score conceals exactly that case, since it trades a temporal error against content that is otherwise right.

Where a unit did not exist at τ (before its inserting amendment, or after its repealing one) we write $g(q) = \emptyset$ and the correct response is a refusal. In NEPVERSA $\emptyset$ is not a rare edge case: it is the correct answer for 61.3% of items (the Results section).

3.2 Corpus

3.2.1 Source selection. Nepali statutes are published in three forms, and only one is usable for a benchmark that requires verbatim quotation.

The Nepal Law Commission (lawcommission.gov.np) publishes consolidated Acts as PDFs. We attempted extraction from all 61 available Act PDFs using both pypdf and PyMuPDF. Every one produced orthographically impossible Nepali: मिति (“date”) extracts as मममि, कार्यान्वयन (“implementation”) as कार्यान्वयि. The cause is not scanning but font encoding. The typesetting pipeline maps legacy Nepali glyphs onto incorrect Unicode codepoints, lossily. The rendered pages are legible; the extracted text is not recoverable by any repair table we could construct, because the mapping is many-to-one.

The Nepal Gazette (राजपत्र) publishes commencement and amendment notices, which is where validity windows would normally come from. Of 53 gazette documents examined, **50 contained no machine-readable text layer**, and the majority date from BS 2011–2013 (1950s CE) rather than the amendment period of interest.

We therefore harvest from **nepallaws.com**, which serves consolidated Acts as structured HTML with one page per provision and the citation hierarchy encoded in the URL path. As of our retrieval date (2026-09-19) the site carried **59 act entries (29 in Nepali and 30 in English)**, the two sets overlapping in content. Text extracted from this source is NFC Devanagari and passes character-exact quotation.

3.2.2 Amendment provenance. Consolidated Nepali Acts carry amendment provenance inline, as superscript footnote markers attached to the unit each amendment touched, with a legend at the foot of the provision (Table 1):

Table 1. Amendment Operation Vocabulary in Both Published Editions

Nepali legend	English legend	Operation
संशोधनद्वारा थपा	Added by the ... Amendment.	INSERT
संशोधनद्वारा संशोधित।	Amended by the ... Amendment.	SUBSTITUTE
संशोधनद्वारा झिकिएको।	Taken out by the ... Amendment.	REPEAL

Legends name the amendment in one of two ways. Most name an **ordinal** (“Added by the First Amendment”). Others name the **amending instrument** (“Inserted by the Financial Act, 2075”, “Amended by the Act to Amend Certain Nepal Acts, 2074”), which is the norm in the Income Tax Act, where amendments arrive through annual Finance Acts. The legend’s own wording is kept as the amendment’s event label in both cases, and the operation is read from its verb. Two irregular forms also occur and are handled explicitly rather than absorbed: a legend with no verb at all (“1 The first amendment.”, throughout the Companies Act), whose operation is read from the units it marks: REPEAL when every one is printed as an elision run, INSERT when every one carries a label that exists only by insertion (1A, u1), and otherwise the provision is rejected rather than guessed; and a misspelt verb in the source (“Extreted from the Financial Act, 2080”), kept verbatim in the vocabulary as a REPEAL, which the elided unit it marks corroborates.

Five inline marker forms occur across the five Acts:

- **Bracketed**, for lettered clauses: `<sup>1</sup>(d1)`
- **Bracketed with a space**: `<sup>1</sup> (b)`
- **Numbered**, for sub-sections: `<sup>1</sup>1)`
- **Lettered without a bracket**: `<sup>2</sup>c)`, `<sup>1</sup>h1)`, `<sup>3</sup>4a)`
- **Label first**, marker after it: `4) <sup>1</sup>.....`

A REPEAL marker counts as removing its unit only when the unit’s text is entirely an elision run. Where words are omitted from a clause that otherwise survives (a `<sup>1</sup>..... theft of the enterprise’s property`), the clause still exists, and recording it as repealed would make every question about it answer “absent”; such markers are left unattached and reported.

Markers are read from the `<sup>` elements of the page HTML, which state them explicitly, and aligned to the Markdown text by position. The Markdown alone is insufficient: it flattens `<sup>2</sup>8)` into 28), which is also how an unmarked sub-section 28 prints (the Results section quantifies the difference). A unit is recorded with the nearest numbered unit above it, so that

clause (c) of sub-section 3 and clause (c) of sub-section 4 of the same section, both repealed in the Income Tax Act, remain two units rather than one.

A repealed unit leaves a positional trace (a run of elision dots where its text stood), so the corpus records not only that something was removed but where.

A second, independent signal comes from numbering. A section inserted by amendment carries a letter suffix (12A, 14B, १२क, १४ख) because it had to fit between existing numbers. Such a section did not exist before the amendment that created it, which is a validity window with a determined opening bound derived without reference to any footnote. We retain both signals and cross-check them (the Method, Validation section).

3.2.3 Version construction. Each amended unit expands into versions bounded by amendment events. Bounds are **half-open on amendment ordinals**, [from, to), so exactly one version governs any point; a closed upper bound would make both the pre- and post-amendment version match at precisely the boundary a point-in-time question is most likely to probe.

What the source determines differs by operation, and we encode the difference rather than smoothing over it (Table 2):

Table 2. What a Consolidated Edition Determines About Each Period

Operation	Prior period	Post period
INSERT	ABSENT (determined)	text printed (determined)
REPEAL	existed, text not printed	ABSENT (determined)
SUBSTITUTE	existed, text not printed	text printed (determined)

The consolidated edition prints only current text. For REPEAL and SUBSTITUTE the superseded wording is therefore **not in the document**, and we mark those periods PRESENT_TEXT_UNKNOWN. No benchmark item is generated that would require reproducing text we do not hold. This is enforced in code, not by convention: the item generator returns nothing for such a period.

Where a legend names an instrument rather than an ordinal, the source does not say where that instrument falls in the Act’s sequence of amendments. Bounds are then **unit-local**: the two versions are “before” and “after” the named instrument, and a question is phrased in the same terms (“before the Financial Act, 2075”), never as “before any amendment”, which would be false for a clause whose Act had earlier, unrelated amendments. Nothing orders one unit’s named instrument against another unit’s ordinal.

3.2.4 Dates. A footnote names an amendment **ordinal** (“First Amendment”) or an amending instrument with the Bikram Sambat year in its title (“Financial Act, 2075”), never a commencement date. The year in an instrument’s title is the year of its enactment, not necessarily the day its amendment took effect, so it is recorded as part of the event label and not converted into a bound. Validity windows are therefore ordinal- or event-valued by default. A Bikram Sambat or Gregorian date is attached only where the corresponding amending Act’s commencement provision has been independently verified; otherwise the date fields remain null. **No date is interpolated.** Bikram Sambat is a lunisolar calendar whose month lengths are tabulated rather than computed, so date arithmetic uses a tested conversion over the supported range BS 1975–2100.

3.3 Benchmark construction

Figure 2 summarises the construction pipeline end to end, with the yield measured at each stage.

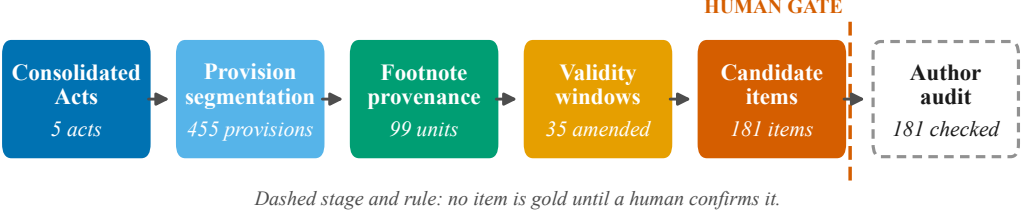


Fig. 2. NEPVERSA Construction Pipeline, With the Yield Counted at Each Stage. Each stage is annotated with the count it produced, taken from paper/shared/numbers.json. The dashed stage and the vertical rule mark the human verification gate: candidate items are generated automatically, but no item is marked verified until a human has checked it against its source. In this release 181 of 181 items were checked by the author (not legally trained); an audit by a reader with legal training is outstanding.

3.3.1 Question types. NEPVERSA defines five types. This release populates T2 and T3, which are the types derivable from amendment footnotes alone; T1, T4 and T5 are defined for completeness and discussed in the Limitations section.

- **T1 Lookup.** Plain retrieval, no temporal component.
- **T2 Point-in-time.** Which version governed at a given point.
- **T3 Supersession.** Whether a unit still governs; the correct answer may be a refusal.
- **T4 Statute-to-case.** From a provision to the judgments interpreting it.
- **T5 Cross-act.** Requires combining provisions from more than one Act.

3.3.2 Gold derivation and scoring. Gold answers are **derived from published statements**, not authored. Where a footnote says a clause was added by the First Amendment, the gold answer for the prior period is that the clause was absent; the provenance string recording that footnote travels with the item.

Scoring uses deterministic **nuggets**: a closed label set, an exact citation key, or a verbatim span. Following Cymbler et al. [2026], no language model judges an answer, since a judge would bring its own temporal errors into the measurement of temporal error. Label and citation nuggets match case-insensitively; **exact nuggets do not**, since they exist to pin verbatim statutory wording.

Formally, an item carries a nugget set $N = \{n_1, \dots, n_m\}$ with each n_j of kind $\kappa(n_j) \in \{\text{label}, \text{citation}, \text{exact}\}$, and a response satisfies the item when every nugget matches:

$$\text{MC}(r) = \prod_{j=1}^m \mathbb{1}[\mu_{\kappa(n_j)}(r, n_j)], \quad \mu_{\kappa}(r, n) = \begin{cases} \text{casefold}(n) \subseteq \text{casefold}(r), & \kappa \in \{\text{label}, \text{citation}\}, \\ n \subseteq r, & \kappa = \text{exact}. \end{cases}$$

The product, rather than a mean, is deliberate: a response that cites the right provision but misquotes it scores zero, which is the behaviour a citation a lawyer will rely on demands. The asymmetry in μ is equally deliberate, and it is the reason the project keeps two normal forms strictly apart. Case-folding a verbatim span would let a near-miss quotation pass the check that exists to guarantee character-exact quotation.

Following Sobhani et al. [2026] we report **two scores that are never averaged**:

- **meaning-correct:** the answer satisfies the content nuggets;
- **time-correct:** the answer abstains where the governing version is absent.

Averaging them would conceal precisely the failure the benchmark exists to expose: a fluent answer quoting a real provision that did not govern at the queried time.

3.3.3 Status of items. All 181 items were checked by a single auditor, the author (BCA, Tribhuvan University; not legally trained). For each item the auditor opened the source page, read the licensing footnote and the provision, and recorded one of *correct*, *incorrect* or *unsure* in an exported audit sheet (AUDIT/audit_sheet_v2.csv, applied by analysis/audit_sheet.py, which refuses any row whose item changed after export). The outcome was 181 correct, 0 incorrect and 0 unsure; items marked correct carry status=verified, and the log (AUDIT/audit_log.json) records the auditor, the stated qualification and the date.

Two limits follow and we state them plainly. First, with one auditor there is no inter-annotator agreement to report, and the auditor also built the pipeline, so the check is not independent. Second, the audit establishes that each item faithfully transcribes its published footnote; it does not establish that a reader with legal training would agree that the item asks a well-formed legal question. An independent audit by such a reader, following the protocol in AUDIT/human_verification_required.md, remains the most valuable next step.

3.4 Validation

Extraction correctness cannot be assessed by inspecting the output, because the output of a partially-correct parser is indistinguishable from that of a correct one: a missed amendment looks exactly like a provision that was never amended. We therefore validate in two ways.

Against markup. Markdown conversion flattens the superscript that carries the footnote marker, so `<sup>1</sup>6)` becomes `16)`, which is shape-indistinguishable from sub-section 16. Markers are therefore read from the published HTML, where `<sup>` delimits them explicitly, and a Markdown-only reading of every page is kept as a comparison: where the two routes disagree, the disagreement is counted and reported (the Results section).

Self-consistency. The benchmark build refuses to write a release in which two items share an identifier, or in which one unit, question type and point in time carries two different gold states. Each item can be individually well-formed while a pair is impossible, and only a check over the whole set sees the pair.

Item audit. A separate audit (analysis/audit_items.py) re-derives each item's gold from the corpus rather than from the generator: every verbatim nugget must occur character-exactly in the provision it cites, every citation nugget must equal the item's key, abstention must coincide with an absent gold state, and each point-in-time gold state must follow from the unit's operation and the side of the amendment the question asks about (an insertion is absent before and present after; a repeal is absent after; a substitution is present after). Run against the earlier 1.0.0 release, it flags every defective item described in the Limitations section; run against this release, it reports none.

Cross-signal. An automated validator compares three signals per provision (inline markers, parsed amendment units, and section numbering) and reports every disagreement, plus elision runs without a recorded repeal and footnotes attached to no unit. Findings are reported as questions rather than failures, because some are genuine properties of the source (the Results section).

3.5 Reproducibility

Every provision record stores its source URL, retrieval timestamp, and the SHA-256 of its extracted text. Identity is the citation key `act:<slug>#<number>` rather than the URL: the site's URL slugs changed during this study and the earlier forms now return 404, so a URL is not a stable identifier.

Harvesting is paced below the source API's measured rate limit, and each run emits a report stating its own validity: a run with any transport failure is marked invalid rather than reported

as a partial success. Raw responses are cached so that re-parsing after a parser change costs no further requests.

4 Results

All figures in this section are produced by `analysis/make_tables.py` from the released corpus and benchmark files and recorded in `paper/shared/numbers.json`; the manuscript’s numbers are substituted from that file at build time. No value is entered by hand.

4.1 Corpus composition

Table 3 reports the corpus as counted from the harvested records, and Figure 3 shows the same counts by Act alongside the amendment operations and item types they yield.

Table 3. Corpus Composition by Act

Act	Provisions	Amended	Density (%)	Lettered	Repealed in full	INS	SUB	REP
Banking Offence and Punishment Act, 2064	34	6	17.6	5	1	6	13	2
Bonus Act, 2030	29	3	10.3	1	5	1	2	3
Companies Act, 2063	190	4	2.1	3	0	3	0	6
Foreign Exchange (Regulation) Act, 2019	46	3	6.5	21	1	1	1	2
Income Tax Act, 2058	156	19	12.2	13	1	36	8	15
Total	455	35	7.7	43	8	47	24	28

Note. “Amended” counts provisions carrying at least one clause-level amendment footnote. “Lettered” counts sections whose number carries a letter suffix, indicating insertion by amendment. INS, SUB and REP are counts of individual amendment operations (insertion, substitution, repeal), of which there are 99 in total. Every harvest run retrieved every discovered page with no transport failure; the pages rejected by the quotability gate are two Schedules of the Income Tax Act and the Companies Act’s Preamble, which carry no section heading, and one Foreign Exchange page with an empty body.

4.2 Amendment style varies sharply between Acts

Amendment density varies roughly eightfold across the five Acts (Figure 4), and the Acts differ in *how* they were amended, not merely *how much*.

For an Act A we report the footnote-visible density

$$\delta(A) = \frac{|\{u \in A : u \text{ carries an amendment footnote}\}|}{|A|},$$

which is what an extraction pipeline tuned to clause footnotes can recover. It is a **lower bound** on how heavily an Act was amended, not a measurement of it, because an Act may record its

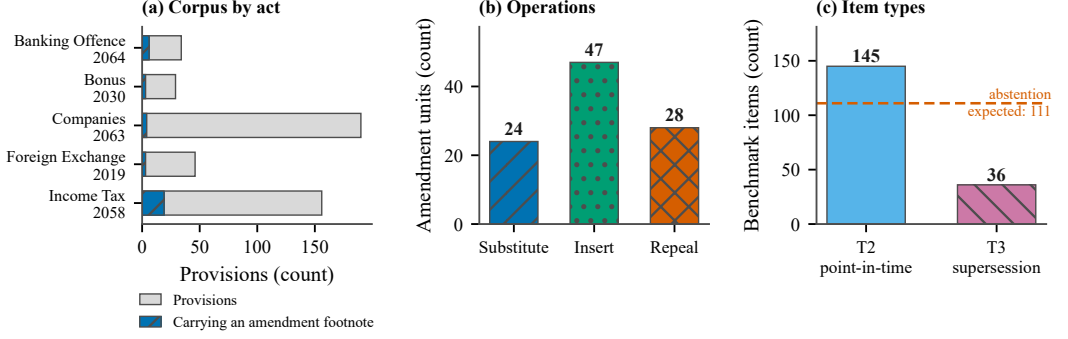


Fig. 3. Composition of the NEPVERSA Corpus and Benchmark. Panel (a) shows provisions per Act, with the shaded and hatched portion counting those that carry at least one clause-level amendment footnote. Panel (b) counts individual amendment operations across all five Acts. Panel (c) counts benchmark items by question type; the dashed rule marks the number of items for which abstention is the correct response. Counts are produced by `analysis/make_tables.py` and plotted by `analysis/make_figures.py` from `paper/shared/numbers.json`. Series are distinguished by hatch as well as colour so the figure survives greyscale reproduction.

amendments structurally instead. Writing $\lambda(A)$ for the count of amendment-inserted (lettered) sections, the selection rule applied to every probed Act is

$$\text{keep } A \iff (\text{amendment units in } A \geq 3) \vee (\lambda(A) \geq 1),$$

and every probed Act is reported with its measured δ and λ , including those that fail the rule. A scope narrowed by an unreported filter is a scope nobody can reproduce.

The Banking Offence and Punishment Act, 2064 is amended predominantly at clause level, and its density is the highest in the corpus (17.6%). It carries 21 amendment operations and 5 sections inserted by amendment (12A, 14A, 14B, 19A, 19B), and every footnote in the harvested edition attributes to the First Amendment.

The Bonus Act, 2030 shows a second pattern: **5 of its 29 sections are repealed in their entirety**, and its footnotes attribute to amendments as late as the Fourth and to two named amending Acts.

The Foreign Exchange (Regulation) Act, 2019 shows a third. It carries only 3 provisions with clause footnotes, but **21 of its 46 sections are amendment-inserted by numbering** (12A, 12B, and so on). At BS 2019 (1962 CE) it is the oldest Act in the corpus, and its amendment history is recorded almost entirely in section numbering rather than in footnotes.

The Income Tax Act, 2058 is the most heavily amended by operation count, 59 of the corpus’s 99, and it is amended in a fourth way: through annual Finance Acts. Its legends name the instrument (“Inserted by the Financial Act, 2075”) rather than an ordinal, and most of its operations are insertions (36) of new sub-sections and clauses numbered to fit between existing ones (1A, 3G, aad1).

The Companies Act, 2063 is the largest Act in the corpus (190 provisions) and the least amended by density (2.1%). Its footnotes are the least regular: every one reads “The first amendment.” with no verb, so the operation is read from what the text shows (the Method section).

This has a practical consequence for corpus construction. An extraction pipeline tuned to clause-level footnotes would suggest the two oldest Acts are barely amended. They are not: their amendments are recorded structurally, as a lettered section number or a section retained with its text

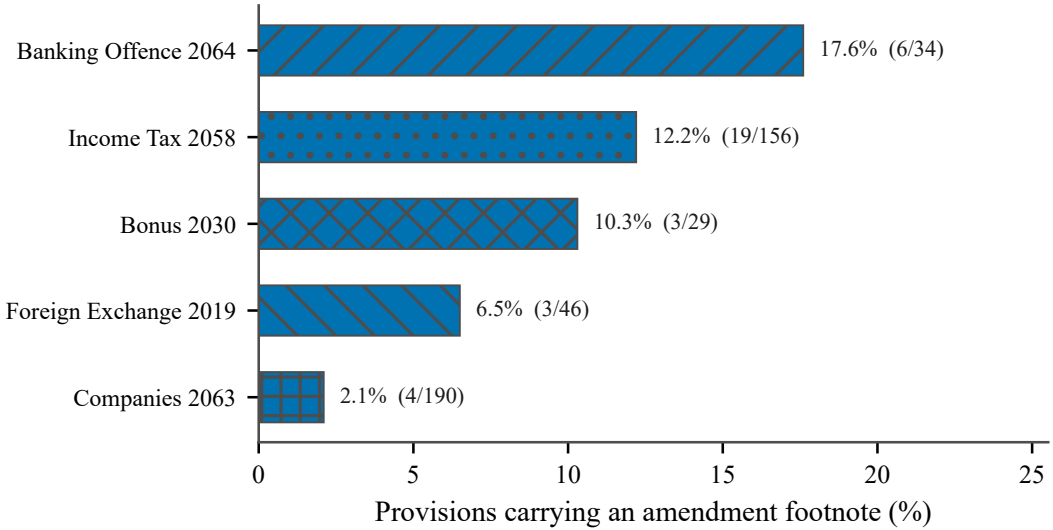


Fig. 4. Amendment Density by Act. The percentage of provisions carrying at least one clause-level amendment footnote, with the underlying counts beside each bar. Density is a measured property of an Act, not an assumption, and it is what decides whether an Act contributes point-in-time and supersession items at all. Bars are distinguished by hatch as well as colour for greyscale reproduction.

replaced by an elision run, rather than in clause footnotes. **Both mechanisms must be handled, or an Act’s amendment history is systematically understated by the pipeline that measures it, and understated most for the oldest Acts, which have the longest history to record.**

4.3 Benchmark composition

Table 4 reports the benchmark by question type.

Table 4. Benchmark Composition by Question Type

Question type	Items	Abstention expected
T2 point-in-time	145	75
T3 supersession	36	36
Total	181	111 (61.3%)

Note. Abstention-expected items are those for which the governing version is absent, so that the correct response is a refusal rather than a citation. Gold is scored by deterministic nuggets: 111 label nuggets, 70 citation nuggets and 70 verbatim-span nuggets across the set. 98 of the T2 items are anchored to a named amending instrument rather than an ordinal. The author’s audit marked 181 of 181 items correct against their source footnotes.

More than half of NEPVERSA items (61.3%) have abstention as the correct response. This is by construction rather than by design choice: an inserted clause did not exist before its inserting amendment, and a repealed clause does not exist after its repealing one, so a question anchored to those periods has no governing text to cite. The share rose from 47.8% in the three-Act release

because the Income Tax Act is dominated by insertions. It positions the benchmark directly on the behaviour Sobhani et al. [2026] found hardest: recognising that a supplied version does not govern, at 26.7% accuracy, against 59.7% for selecting one that does.

4.4 Extraction accuracy

Amendment markers are read from the <sup> elements of the published HTML. Two checks bound the error of that route.

Every marker is accounted for. The five Acts’ pages carry 119 inline markers. Of these, 99 are attached to a parsed amendment unit, 8 mark a section repealed in its entirety, and 12, in 6 provisions, attach to nothing. Each of those is reported by the validator and was inspected against the page: they mark amendments *inside* a sentence or a proviso (for example, words substituted mid-clause in the Bonus Act’s definitions), which a clause-level corpus cannot represent, or carry a marker number with no legend at all (Income Tax §95A marks a sub-section with a footnote 3 that the page never defines). None is silently dropped, and no unit is invented for them.

The Markdown route, measured. Table 5 re-reads the same cached pages with markers inferred from the Markdown alone, which is how the first release of this corpus was built.

Table 5. Amended Units Found by the HTML and Markdown-Only Routes

Act	Units (HTML)	Units (Markdown only)	Agree	Markdown only	HTML only	Provisions disagreeing
Banking Offence and Punishment Act, 2064	21	21	21	0	0	0
Bonus Act, 2030	6	3	3	0	3	3
Companies Act, 2063	9	9	9	0	0	0
Foreign Exchange (Regulation) Act, 2019	4	4	4	0	0	0
Income Tax Act, 2058	59	42	36	6	23	9
Total	99	79	73	6	26	12

Note. “Markdown only” are units the Markdown reading reports and the HTML does not (invented); “HTML only” are units the Markdown reading misses. Counted by analysis/make_tables.py over the cached pages of every provision in the corpus.

On the Banking, Foreign Exchange and Companies Acts the two routes agree exactly. In the Bonus Act the Markdown reading misses 3 units, and the Income Tax Act breaks the inference in both directions (6 invented, 23 missed). The Markdown reading **invents** units wherever a sub-section is numbered ten or above under a legend numbered 1 (10) read as footnote 1 governing a sub-section 0), which does not exist), and it **misses** units whose label is an upper-case insertion (¹2A)), a three-letter clause (aad1), or is printed before its marker (4) ¹..... in the Bonus Act). Overall it reports 79 units where the HTML states 99, with 6 invented and 26 missed, in 12 provisions.

The cross-signal validator reports its remaining findings as questions, not failures, and they are properties of the source:

- **Lettered sections without footnote provenance.** Sections identifiable as amendment-inserted from their numbering, but whose pages carry no corresponding footnote. The insertion is real and its provenance is recorded only structurally, which is why the pipeline keeps numbering as an independent second signal.
- **Orphan footnotes and unexplained elisions.** A legend that names an amendment but attaches to no marked unit, and a few elision runs with no legend at all (Companies §184 prints sub-section 3) with no footnote). Each is retained as found; nothing is invented to explain it.

4.5 Extraction defects found during construction

Fifteen distinct defects were found and fixed while building this corpus, seven on the first three Acts and eight more when two further Acts were added. We report them because their distribution is itself a finding: **none produced a visible symptom in the output** (Table 6).

Table 6. Extraction Defects Found During Construction, and How Each Was Found

#	Defect	How found
1	Repeal phrasing Taken out by the ... Amendment. absent from the vocabulary	Manual reading
2	Footnote marker with no separating space (1Amended by ...)	Manual reading
3	Wholly repealed section rejected by a minimum-length check	Manual reading
4	Unknown-verb guard triggered on ordinary prose about amendment	Adversarial test
5	Repealed-in-full flag never written to the output records	Cross-signal validator
6	Second inline marker form (¹1)) unhandled	Cross-signal validator
7	Flat-structure Acts yielded zero provisions in discovery	Manual reading
8	Pages with no tag list lacked the assumed end marker; every provision of two Acts rejected	Parse-rejection count
9	Bracketed marker with a space (¹ (b)) unhandled	Per-Act amended count of zero
10	Lettered marker without a bracket (²c)) unhandled	Manual reading
11	Verbless legend (The first amendment.) skipped as an ordinary list item	Manual reading
12	Legend naming an instrument with an unrecognised verb skipped silently	Manual reading
13	Markdown inference reads plain sub-section 10) as a marked unit 0)	Manual reading; HTML comparison
14	Instrument-named amendments left both version bounds open: contradictory gold	Reading released items
15	Identical clause letters under different sub-sections keyed as one unit	Manual reading

Defects 5 and 6 were found by the automated validator after manual inspection had passed the same corpus; defect 5 was introduced by the fix for defect 3. Before defects 1–3 and 5–6 were corrected, the Banking Act yielded 2 amended provisions and 14 benchmark items with no T3

items at all; afterwards it yields 6 amended provisions and 30 items including 3 T3 items. **The uncorrected pipeline reported a complete, internally consistent corpus while recovering under a third of the published amendments.**

Defect 8 had the largest immediate cost: the Income Tax and Companies Acts harvested cleanly, cached cleanly, and yielded zero provisions, because the parser required a tag list that untagged pages do not print. Only the rejection counter showed it. Defect 14 is the one we most want other builders to notice. It reached the public 1.0.0 release, it produced items that were each well-formed, and it was visible only in a pair: the same clause, at the same point, stated to be both absent and present. The build now checks the whole item set for such pairs before writing anything (the Method, Validation section).

Defect 7 is the most consequential for coverage. Acts on the source site follow two structures (provisions nested under chapters, and provisions listed directly) and a discovery routine written for the first returns *zero* provisions for the second, silently and indistinguishably from an Act with no sections.

4.6 Retrieval regime

We re-derived retrieval measurements on an independent Nepali legal corpus of 330 Supreme Court judgments (4,102 passages) using a lexical BM25 retriever [Robertson and Zaragoza 2009] with proximity scoring, over a 31-question labelled set with 2 unanswerable controls: **P@1 = 0.871, Recall@5 = 0.968, MRR = 0.925**, with both unanswerable questions correctly refused and no false refusals.

These figures are consistent in direction with Wagle et al. [2026], who report BM25 at P@1 0.91 against multilingual-E5-large at 0.75 on a different Nepali legal corpus. Two independent measurements now place a lexical baseline above a multilingual dense retriever on Nepali legal text, the inverse of the pattern that adaptive-retrieval methods developed on English benchmarks assume.

We note this as context for interpreting version-aware results in this language, not as a controlled comparison: our measurement and Wagle et al.’s use different corpora and different labelled sets, and we did not run a dense retriever on NEPVERSA.

5 Discussion

5.1 A partially-correct parser produces an apparently-correct corpus

The result we consider most transferable is not a number but a failure mode.

Across the construction of this corpus, fifteen distinct extraction defects were found. Every one had the same observable signature: **none**. A provision whose amendment footnote was missed enters the corpus looking exactly like a provision that was never amended. There is no malformed field, no parse exception, no anomalous value, only a record that is quietly wrong about the one property the benchmark exists to measure.

This is a sharper problem than ordinary extraction noise. A missing *text* field is visible; a missing *amendment* is not, because absence of amendment is itself a valid and common state. The usual coverage check, “34 of 34 provisions parsed, zero rejected”, was true of the corpus at a point when it was recovering fewer than half the amendments actually published in it.

Four mitigations proved effective, in increasing order of value:

1. **Fail loudly on unrecognised vocabulary.** When a footnote names an amendment using a verb the parser does not know, raising is strictly better than skipping. A rejected provision is a visible gap; a skipped footnote is an invisible falsehood. This required narrowing the trigger to footnote *grammar* (“by the First Amendment”) rather than the word

- “amendment”, since statutes contain ordinary prose about amendment that must not be rejected.
2. **Maintain redundant signals.** Amendment-inserted sections are identifiable two ways: from a footnote, and from a lettered section number (12A, १२क). Neither is complete. Sections exist whose insertion is recorded only in the numbering, and footnotes exist that name no unit at all.
 3. **Cross-check the signals automatically.** A validator comparing inline markers, parsed amendment units, and section numbering, reporting every disagreement, found defects that manual inspection had missed, including one in the code written to fix earlier defects. We report its findings as questions rather than failures, because a fraction are genuine properties of the source.
 4. **Check the item set, not only the item.** Every contradictory item in the 1.0.0 release passed every per-item check; the contradiction existed only between two items about the same clause. A build-time check that one unit, question type and point in time carries exactly one gold state now refuses such a release before anything is written.

5.2 Format fidelity determines which facts survive

The published HTML marks an amended unit with a superscript: `<sup>1</sup>1`) If any-one commits.... Markdown conversion flattens the superscript into the surrounding text, so `<sup>1</sup>6`) becomes 16), shape-indistinguishable from sub-section 16.

The first release of this corpus read markers from Markdown by inference, and it was checked against the HTML on the Act where that seemed hardest, the Banking Act, where the two agree exactly. Over five Acts the Markdown-only reading finds 79 amended units where the HTML states 99; it invents 6 and misses 26, in 12 provisions (Table 5), including 3 missed repeals in the Bonus Act, which was part of that first release. Every invented unit comes from a sub-section numbered ten or above (a plain 10) under a legend numbered 1 reads as footnote 1 governing a sub-section 0), which does not exist), and such sub-sections occur only in the longer Acts. An inference validated on one short Act was validated on exactly the inputs where it cannot fail.

The general lesson for corpus construction in this setting: **the serialisation you scrape decides which facts survive it.** A conversion that preserves prose perfectly can destroy exactly the markup that carries provenance, while leaving text that still reads correctly, and therefore passes every check that inspects text.

5.3 Implications for version-aware retrieval in low-resource settings

Cymbler et al. [2026] show that, over French tax law, retrieval from the current text alone never finds the version that applied at the date asked about, and that indexing every version largely removes the problem. Sobhani et al. [2026] show that recognising a non-governing version is far harder than selecting a governing one. Both results presume a retrieval stage that reliably surfaces candidates.

That premise is weaker in Nepali. Wagle et al. [2026] report BM25 outperforming multilingual-E5-large by 16 points on Nepali legal retrieval, and our independent measurements on a different Nepali legal corpus are consistent in direction. Where first-stage recall is the binding constraint, version-selection machinery may not be reached at all, which would make the *ordering* of interventions different from the high-resource case: corpus versioning and lexical retrieval quality before agentic version selection.

We state this as a hypothesis that NEPVERSA makes testable, not as a result. The present work supplies the resource; the comparison is future work (the Future Work section).

5.4 What the source cannot tell us

Consolidated editions print current text. For a substituted or repealed unit the superseded wording is therefore absent from the document, and we mark those periods as textually undetermined rather than imputing content.

This bounds the benchmark in a specific and, we would argue, honest way. Insertions and repeals yield fully determined validity windows and support questions of both kinds: *what governed* and *whether anything governed*. Substitutions support only the latter. A benchmark that asked models to reproduce superseded wording we never observed would score every system wrong for a reason attributable to the corpus rather than the system.

Similarly, footnotes name amendment *events* (an ordinal, or an amending instrument such as “Financial Act, 2075”), not commencement dates. Validity windows are event-valued unless a commencement date has been independently verified. The year in an instrument’s title is tempting to use as a date, and we do not: it is the year the instrument was enacted, which need not be the day its amendment took effect. We leave date fields null rather than interpolating, because an interpolated date propagates silently: every downstream check can be internally consistent and uniformly wrong about which year applies.

6 Limitations

Scale. This release covers a small number of Acts. The extraction method is not the constraint; retrieval quota is. Amendment density varies substantially across Acts, and the corpus is weighted toward Acts selected for expected density rather than sampled representatively. Aggregate amendment rates reported here should not be read as estimates of amendment rates in Nepali legislation generally.

Question-type coverage. Only T2 (point-in-time) and T3 (supersession) items are populated. These are the types derivable from amendment footnotes alone. T1 (lookup) requires no additional evidence and is straightforward to add; T4 (statute→case) requires linkage to judgment text; T5 (cross-act) requires multiple Acts in a single reasoning chain. Their definitions are included so that later releases extend rather than redefine the benchmark.

No date-anchored windows. Validity windows are bounded by amendment events (“First Amendment”, or a named instrument such as “Financial Act, 2075”), because the footnotes name events and not commencement dates. Where the event is a named instrument, its bounds are unit-local: the release says a clause governed before or after that instrument, but not where the instrument falls among the Act’s other amendments, so questions spanning two units amended by differently named events cannot be generated. Attaching Bikram Sambat and Gregorian dates requires each amending Act’s commencement provision, which the gazette record does not supply in machine-readable form. Consequently the benchmark tests *version* resolution but not yet *date* resolution, which is the form the question takes in practice (“what was the law on 2078-03-15?”). This is the single largest gap between NEPVERSA and comparable resources such as [Cymbler et al. \[2026\]](#), whose corpus carries real date bounds.

Superseded text is unavailable. Consolidated editions print only current text. For substituted and repealed units the prior wording is not in the document, so those periods are marked textually undetermined and generate no quotation questions. A benchmark built from *amending Acts* rather than consolidated editions would recover this, at the cost of a much harder alignment problem.

No model evaluation. This paper reports a resource and an extraction evaluation. It does not evaluate retrieval or generation systems on NEPVERSA. The claims in the Results section concern extraction correctness, not model capability, and nothing here should be read as a finding about how models perform on Nepali version-aware retrieval.

The scorer rewards containment, and containment is not comprehension. The nugget scorer is deliberately deterministic, which buys reproducibility and costs discrimination. We tested it adversarially against the output patterns deployed models actually produce (tests/test_scoring_adversa and record four properties that a user of this benchmark must know.

Negation is invisible. A nugget is satisfied by substring containment, so “section 5 does not apply” scores identically to “section 5 applies”. The same holds in Nepali, where negation is a verb ending (लागू हुँदैन) carrying no separable negative token. This project has twice shipped bugs of exactly this shape, and we expect it to be the scorer’s largest source of false credit.

Hedging is free. An answer prefixed with “I am not certain, but” scores the same as a confident one, so the benchmark cannot distinguish calibrated uncertainty from assertion.

Abstention detection is keyword-based, and under-detects. An honest refusal phrased outside the marker list (“that clause had not yet been inserted”) is scored as a failure to abstain. Because 61.3% of items are abstention-expected, this penalises precisely the behaviour the benchmark was built to reward, and a reported abstention score is therefore a **lower bound** on a system’s real abstention ability. Conversely, one marker sentence inside a confident wrong answer satisfies the check, so the measure is gameable in both directions.

Recall is rewarded over selection. An answer listing many plausible citations satisfies a citation nugget as surely as one that names only the governing provision.

None of these is repaired by making the scorer cleverer, because every plausible repair reintroduces a second normal form or a model in the judging loop, which is what the deterministic design exists to avoid. They are stated here so that a score on NEPVERSA is read as what it is: a necessary condition that a correct system passes, not a sufficient one that only a correct system passes.

The audit is a single, non-expert, non-independent check. This is the most important limitation of the release. All 181 items were checked against their source footnotes by the author, who is not legally trained and who also built the pipeline; all were judged correct. There is therefore no inter-annotator agreement to report, and the check confirms fidelity of transcription rather than legal well-formedness. A reader should treat status=verified as meaning *checked against the published footnote by the author*, not as endorsement by a legal professional. AUDIT/human_verification_require specifies the protocol, stratified sample and agreement statistics for an independent audit by a reader with legal training, which we invite.

An earlier release contained contradictory items. Release 1.0.0 of this benchmark, published before this paper, contained items generated from amendments named by instrument rather than ordinal. Their validity windows were left unbounded on both sides, and four of its 46 items were wrong as a result: one clause was stated to be both absent and present “before any amendment”, a substituted clause’s new text was given as its text before the substitution, and a clause that was later repealed was stated to have been absent before any amendment. Release 2.0.0 corrects all four; the build now refuses any release in which one unit and point in time carries two gold states, and the item audit flags all four when run against the old release. The affected item identifiers are listed in the release changelog. We report this because it is the failure mode the rest of the paper describes, occurring in our own output: every affected item was individually well-formed.

Single source. All statutory text comes from one aggregator (nepallaws.com). Its consolidations were not independently reconciled against the Nepal Law Commission’s own texts, because those are the PDFs whose extraction failure motivated the choice of source (the Method, Source Selection section). Errors in the aggregator’s consolidation would propagate into the corpus undetected by any check described here.

English and Nepali editions. The site publishes both. Amendment footnotes appear in both, and the extraction vocabulary covers both, but this release does not verify that the two editions

agree provision-by-provision on which units were amended. Disagreement between them would be an informative signal and is left to future work.

7 Ethical considerations

Licence and redistribution. The licence status of Nepali statutory text is unresolved. Government legal texts are typically freely reproducible, but “typically” is not a licence, and no explicit terms are published by the source aggregator. We therefore release **identifiers, character offsets, amendment annotations, nugget gold, and a rebuild script**, not the statutory text itself. Anyone can reconstruct the corpus by running the harvester against the live source. Where licence terms are later clarified, a full release becomes possible without changing the benchmark.

Personal data. Statutory text contains no personal data. The judgment corpus referenced for future T4 items consists of published court reports; no additional personal information is introduced beyond what those reports already make public.

Intended use and misuse. NEPVERSA is a research resource for evaluating retrieval systems. It is not legal advice, and its items are not a substitute for professional judgment. A system scoring well on it has demonstrated version resolution on a specific corpus, not competence in Nepali law. We note the asymmetry deliberately: in legal practice a confident wrong citation is more damaging than an acknowledged gap, which is why abstention is measured as a first-class outcome rather than penalised as non-response.

Competing interests. The author develops a commercial legal research tool for Nepali advocates. This research was conducted in the same repository and reuses that project’s Nepali language-processing components (text normalisation, stemming, romanisation, and Bikram Sambat date arithmetic); the corpus, extraction method, benchmark and evaluation reported here are new work. The relationship is disclosed in full in the Author Note.

8 Use of generative AI tools

Generative AI tools were used for literature discovery, code assistance, and drafting [Anthropic 2026]. All extraction code was reviewed and tested by the author; every reported number is produced by a script in the released repository and recorded with its source and generating command. No benchmark item, gold answer, citation, or measurement in this paper was generated by a language model. Citations were verified against the arXiv API and, for published versions, against Crossref by DOI, with author lists taken from those responses rather than from model output; this process corrected two attribution errors present in earlier drafts. The author reviewed and edited all content and takes full responsibility for it.

9 Conclusion

Version-aware legal retrieval has been shown to fail in ways that ordinary retrieval evaluation cannot see: a system returns a real provision, correctly cited, that does not govern the period in question [Cymbler et al. 2026], and recognising non-applicability is markedly harder than selecting an applicable version [Sobhani et al. 2026]. These results come from languages with mature retrieval infrastructure. Whether they transfer to a low-resource language whose strongest retriever is lexical has not been testable, because no version-aware corpus existed in such a language.

We have supplied one. NEPVERSA is built from a source of amendment provenance that is available where the conventional source is not: the numbered footnotes that consolidated Nepali Acts carry inline, which name the amending instrument and its operation at clause granularity. Gold answers are therefore *derived from published statements* rather than authored, which makes the resource constructible without large-scale expert annotation and makes every item auditable against a specific sentence in the law.

We are explicit about the resulting bounds. Insertions and repeals yield fully determined validity windows; substitutions determine the post-amendment text only, and we mark the prior period undetermined rather than impute it. Footnotes name amendment events (ordinals or amending instruments) rather than commencement dates, so windows are event-valued until commencement dates can be independently verified. These are properties of what Nepal publishes, not of the method. Recording them honestly, and stating exactly who checked each item and with what qualification, is what makes the resource usable by someone who did not build it.

The methodological finding we expect to transfer furthest concerns extraction itself. **A footnote parser that is approximately correct produces a corpus that appears entirely correct**, because a missed amendment is indistinguishable in the output from a provision that was never amended. Coverage statistics cannot detect this; only redundant signals, loud failure on unrecognised vocabulary, and automated cross-checking can. Any project deriving provenance from consolidated legal text, in any jurisdiction, inherits this problem, and inherits it silently.

9.1 Future work

Date anchoring. Attaching Bikram Sambat and Gregorian bounds to validity windows requires the commencement provision of each amending Act. Obtaining these in machine-readable form is the highest-value next step, and is primarily an access problem rather than a technical one.

Model evaluation. NEPVERSA makes testable the hypothesis raised in the Results section: that where first-stage recall is the binding constraint, version-selection machinery may not be reached, reordering the interventions that matter relative to the high-resource case. Evaluating single-pass and agentic systems at matched retrieval budgets is the natural continuation.

Scale and coverage. Extending to more Acts, populating T1, T4 and T5, and reconciling the Nepali and English editions provision-by-provision would strengthen the resource considerably. Edition disagreement in particular is a free validation signal that this release does not yet exploit.

Portability. The extraction method depends on consolidated text carrying inline amendment footnotes, which is a common convention well beyond Nepal. Testing it against another jurisdiction’s consolidations would establish whether the approach generalises or is an artefact of one publisher’s typesetting.

Acknowledgments

The author develops a commercial legal research tool for Nepali advocates. This research was carried out in the same code repository and reuses that project’s Nepali language-processing components (text normalisation, stemming, romanisation, and Bikram Sambat date arithmetic). The corpus, extraction method, benchmark, and evaluation reported here are new work produced for this study. This relationship is disclosed as a competing interest.

Generative AI tools (Claude, Anthropic) were used for literature discovery, code assistance, and drafting; their use is described in the Method section. No benchmark item, gold answer, citation, or measurement was generated by a language model. The author reviewed and edited all content and takes full responsibility for it.

Data, extraction code, and benchmark annotations are available at <https://github.com/NikeGunn/nyasathireserch>. Because the licence status of the underlying statutory text is unresolved, the release comprises identifiers, offsets, annotations, and a rebuild script rather than the source text.

References

Praveen Acharya and Bal Krishna Bal. 2026. Towards Building a Standard Benchmark for Low-Resource Nepali Information Retrieval. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM. doi:10.1145/3805712.3808533

- Anthropic. 2026. Claude. Large language model. <https://claude.ai> Used for literature discovery, code assistance, and drafting; see Method, Use of Generative AI Tools.
- Wenhu Chen, Xinyi Wang, and William Yang Wang. 2021. A Dataset for Answering Time-Sensitive Questions. *arXiv preprint arXiv:2108.06314* (2021). arXiv:2108.06314 doi:10.48550/arXiv.2108.06314
- Rose Cymble, Daniel Guez, and Laurent Fabre. 2026. Temporal Misgrounding in Legal RAG: A Versioned-Corpus Benchmark for French Tax Law. *arXiv preprint arXiv:2608.09393* (2026). arXiv:2608.09393 [cs.CL] doi:10.48550/arXiv.2608.09393 FiscalQA Pro: 32,436 article-versions 1938–2031; deterministic nugget scoring, no LLM judge.
- Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. 2022. Time-Aware Language Models as Temporal Knowledge Bases. *Transactions of the Association for Computational Linguistics* 10 (2022), 257–273. doi:10.1162/tacl_a_00459
- Faizan Faisal and Umair Yousaf. 2024. LEGAL-UQA: A Low-Resource Urdu-English Dataset for Legal Question Answering. *arXiv preprint arXiv:2410.13013* (2024). arXiv:2410.13013 [cs.CL] doi:10.48550/arXiv.2410.13013
- Wei Fan, Yining Zhou, Mufan Zhang, Yanbing Weng, Yiran Hu, and Tianshi Zheng. 2026. Can LLMs Time Travel? Enhancing Temporal Consistency in Legal Agentic Search through Reinforcement Learning. *arXiv preprint arXiv:2605.25920* (2026). arXiv:2605.25920 [cs.CL] doi:10.48550/arXiv.2605.25920 LegalSearch-R1; temporally-indexed training across amendment periods.
- Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, et al. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems* 36. 44123–44279. doi:10.52202/075280-1915
- Abe Bohan Hou, Orion Weller, Guanghui Qin, Eugene Yang, Dawn Lawrie, Nils Holzenberger, Andrew Blair-Stanek, and Benjamin Van Durme. 2025. CLERC: A Dataset for U.S. Legal Case Retrieval and Retrieval-Augmented Analysis Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 7898–7913. doi:10.18653/v1/2025.findings-naacl.441
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*. arXiv:2403.14403 doi:10.48550/arXiv.2403.14403
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active Retrieval Augmented Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. arXiv:2305.06983 doi:10.48550/arXiv.2305.06983 FLARE.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. *arXiv preprint arXiv:2503.09516* (2025). arXiv:2503.09516 [cs.CL] doi:10.48550/arXiv.2503.09516
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. arXiv:2005.11401 doi:10.48550/arXiv.2005.11401
- Adam Liška, Tomáš Kočíský, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, et al. 2022. StreamingQA: A Benchmark for Adaptation to New Knowledge over Time in Question Answering Models. *arXiv preprint arXiv:2205.11388* (2022). arXiv:2205.11388 doi:10.48550/arXiv.2205.11388
- Antoine Louis and Gerasimos Spanakis. 2022. A Statutory Article Retrieval Dataset in French. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6789–6803. doi:10.18653/v1/2022.acl-long.468
- Robert Mahari, Dominik Stambach, Elliott Ash, and Alex Pentland. 2024. LePaRD: A Large-Scale Dataset of Judicial Citations to Precedent. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 9863–9877. doi:10.18653/v1/2024.acl-long.532
- Nicholas Pipitone and Ghita Houir Alami. 2024. LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. *arXiv preprint arXiv:2408.10343* (2024). arXiv:2408.10343 doi:10.48550/arXiv.2408.10343
- Ronak Pradeep, Nandan Thakur, Shivani Upadhyay, Daniel Campos, Nick Craswell, and Jimmy Lin. 2024. Initial Nugget Evaluation Results for the TREC 2024 RAG Track with the AutoNuggetizer Framework. *arXiv preprint arXiv:2411.09607* (2024). arXiv:2411.09607 doi:10.48550/arXiv.2411.09607
- Max Prior, Andreas Schultz, and Matthias Grabmair. 2026. Asking For An Old Friend: Diagnosing and Mitigating Temporal Failure Modes in LLM-based Statutory Question Answering. *arXiv preprint arXiv:2605.23497* (2026). arXiv:2605.23497 [cs.CL] doi:10.48550/arXiv.2605.23497 312 expert-validated German statutory QA pairs; post-cutoff staleness and recency bias.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know What You Don’t Know: Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 784–789. doi:10.18653/v1/P18-2124

- Ranjit Raut, Tishya Dhakal, Aaryan Shakya, Bhabuk Thapa, Prasiddha Koirala, and Bal Krishna Bal. 2026. Nepali Legal Expertise through Generative and Extractive Pre-trained Transformers (NepLEGiT). *arXiv preprint arXiv:2609.16010* (2026). [arXiv:2609.16010](#) [doi:10.48550/arXiv.2609.16010](#)
- Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* (2009). [doi:10.1561/15000000019](#)
- Mahbub E Sobhani, Md. Faiyaz Abdullah Sayeedi, Fahmid Hasan Chowdhury, Md Adnan Arefeen, Farig Sadeque, and Md. Faizul Bari. 2026. Time Present and Time Past: Benchmarking Large Language Models on Temporally Evolving Document Understanding. *arXiv preprint arXiv:2608.08512* (2026). [arXiv:2608.08512](#) [cs.CL] [doi:10.48550/arXiv.2608.08512](#) TIDE: 3,050 QA pairs, 644 Bangladeshi customs instruments 1969–2025; version resolution; dual calendars; code-mixed.
- Bhabuk Thapa, Prasiddha Koirala, Ranjit Raut, Sunil Regmi, and Bal Krishna Bal. 2026. NepKANUN: A RAG-Based Nepali Legal Assistant. *arXiv preprint arXiv:2609.15999* (2026). [arXiv:2609.15999](#) [cs.CL] [doi:10.48550/arXiv.2609.15999](#) Fine-tuned LLM + RAG assistant; BERTScore F1 0.82/0.77/0.71; no amendment or temporal dimension.
- Sulav Timilsina, Milan Gautam, and Binod Bhattarai. 2022. NepBERTa: Nepali Language Model Trained in a Large Corpus. In *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*. Association for Computational Linguistics, 273–284. [doi:10.18653/v1/2022.aacl-short.34](#)
- Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. [arXiv:2212.10509](#) [doi:10.48550/arXiv.2212.10509](#) IRCot.
- Samir Wagle, Abiral Adhikari, Reewaj Khanal, Batsal Bhandari, Prashant Manandhar, Praveen Acharya, and Bal Krishna Bal. 2026. Retrieval Augmented Generation Framework for the Nepali Legal Domain Question Answering. *arXiv preprint arXiv:2606.07523* (2026). [arXiv:2606.07523](#) [cs.CL] [doi:10.48550/arXiv.2606.07523](#) First Nepali legal RAG; NKP case law only; BM25 P@1 0.91 > multilingual-E5-large 0.75; no statutes, no versioning.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. [arXiv:2210.03629](#) [doi:10.48550/arXiv.2210.03629](#)
- Michael Zhang and Eunsol Choi. 2021. SituatedQA: Incorporating Extra-Linguistic Contexts into QA. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 7371–7387. [doi:10.18653/v1/2021.emnlp-main.586](#)